

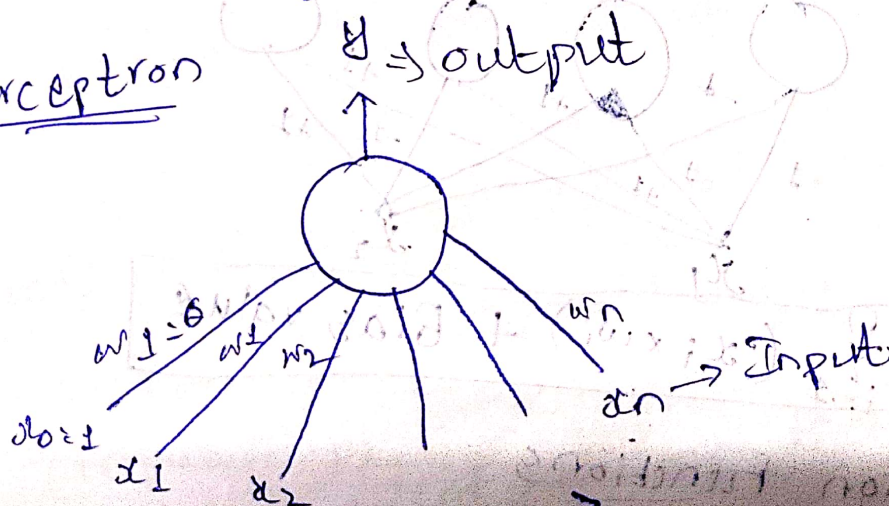
Deep Learning:- Specialized branch of machine learning which uses multi-layered neural networks.

* Each layer transfers the input data into more abstract representations.

ML vs DL

ML	D.L
* Works on small amount of dataset for accuracy.	* Works on large amount of Dataset.
* Dependent on Low-end Machine.	* Heavily dependent on High & Machine.
* Divides the tasks into sub-tasks	* Solves problem end-to-end.
* Taking less time to train	* Takes longer time to train.
* More testing time	* Less testing time

Perceptron



$$y = 1 \text{ if } \sum_{i=0}^n w_i x_i \geq 0$$

$$= 0 \text{ if } \sum_{i=0}^n w_i x_i < 0$$

where $x_0 = 1$ & $w_0 = -\theta$

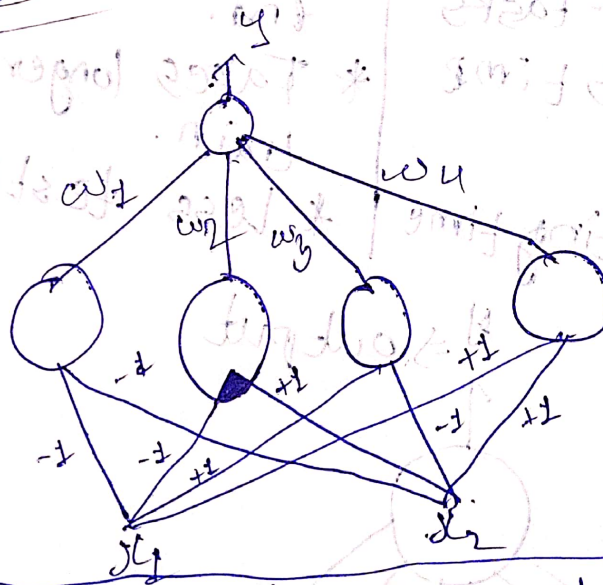
Learning Algorithm (Perceptron)

while convergence do
if $x \in P$ & $\sum_{i=0}^N w_i + x_i < 0$ then
 $w = w + x_i$
end
if $x \in N$ and $\sum_{i=0}^N w_i + x_i \geq 0$ then
 $w = w - x_i$
end
end

*P = inputs with label 1

*N = inputs with label 0

Multilayer Perceptron



$$Y = \sum_{i=1}^N (x_i \times w_i) + \text{Bias value}$$

Activation Functions

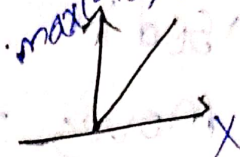
* The activation function determines the output based on weighted sum of the inputs & the bias term.

RELU Activation Function

24/07/20

* Rectified Linear Unit

* non linear activation function frequently employed in neural networks.



$$f(x) = \max(0, x)$$

* ReLU performs better in deep learning than "sigmoid", "tanh".

Leaky RELU Activation Function

* modified version of RELU function.

$$f(x) = \begin{cases} ax & , x < 0 \\ x & , x \geq 0 \end{cases}$$

* Can identify dead neurons.

Softmax Activation Function

* Based on the sigmoid function which yields the outputs as a probability in the range (0,1).

* Combination of multiple sigmoid functions.

$$f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}}$$

Feedforward Neural Networks

* Information flows in one direction (input $\rightarrow$ output).

* Simplest type of ANN architecture.

* Each neuron in a layer are fully connected to all other neurons.



Gradient Descent (iterative)

* Most fundamental optimization algorithm used in deep learning.

* Moves the network parameters by moving them in the direction that reduces the loss function.

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t) \quad \Rightarrow \text{update rule.}$$

θ_t = Current weights of the network.

η = Learning rate.

$\nabla J(\theta_t)$ = Gradient of the loss function.

θ_{t+1} = Updated weights after one iteration.

Challenges in DL optimization

- Local minima
- Saddle point
- Vanishing gradient
- Exploding gradient
- Learning rate overshooting.

Step-By-Step Algorithm

Initialize weights & biases



Perform forward Propagation



Compute the Loss Function



Perform Backpropagation

Compute the Gradient
↓
Update the Parameters
↓
Repeat for multiple Epochs.

Batch Gradient Descent (BGD)

* This uses the entire ~~gradient~~ training dataset before updating the model parameters.

$$\theta = \theta - \eta \nabla J(\theta) \Rightarrow \text{update rule.}$$

Stochastic Gradient Descent (SGD)

* This updates the model parameters using the gradient computed from only one randomly selected training sample.

* One sample $\rightarrow$ One update.

$$\theta = \theta - \eta \nabla L_i(\theta) \Rightarrow \text{update rule.}$$

Mini-batch GD

* This computes using a small random subset.

$$\theta = \theta - \eta \nabla J_{\text{batch}}(\theta) \Rightarrow \text{update rule.}$$

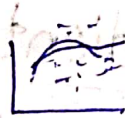
Bias Variance Tradeoff

05/08/26

Bias = A simple model that under-fits the data.

Variance = A complex model that over-fits the data.

High Variance = Overfitting.



High Bias = underfitting.



Less bias, Less Variance = Good balance.



Regularization or Ridge Regularization

* We use this L2 Regularization technique in Deep Learning.

* Used to reduce overfitting.

$$L(x, y) = \sum_{i=1}^n (y_i - h(x_i))^2 + A \sum_{i=1}^n \theta_i^2$$

VGG Architecture

Step Activation function

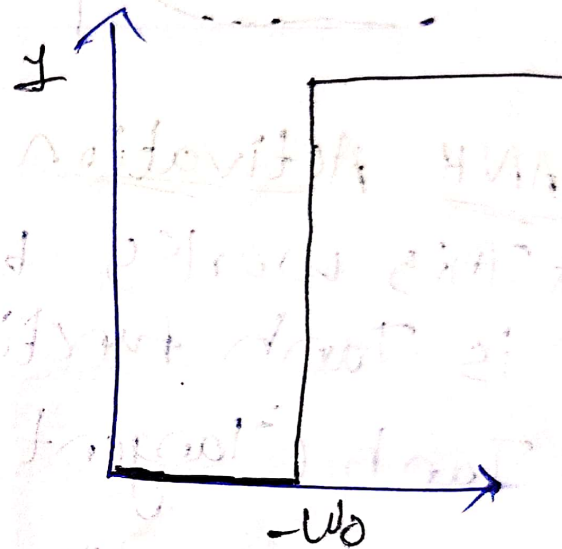
* This compares the input to a threshold.

* Activation occurs if the input exceeds it, otherwise, the neuron gets deactivated.

$$\rightarrow f(x) = 1 \text{ if } x \geq 0$$

$$\rightarrow f(x) = 0 \text{ if } x < 0$$

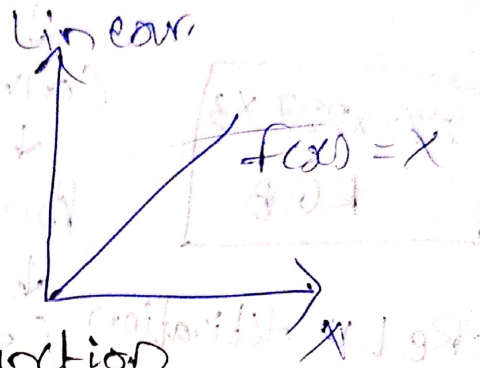
$$z = \sum_{i=1}^n w_i x_i$$



Linear Activation Function

- * Also called the identity function.
- * Just outputs the same value it was given.

$$f(x) = x;$$

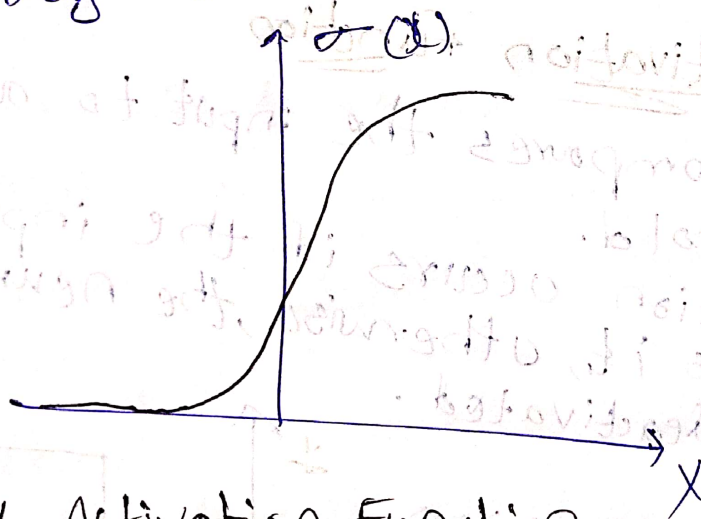


Sigmoid Activation Function

- * This is non-linear.
- * The sigmoid function modifies values between 0 & 1.

$$f(x) = 1 / (1 + e^x)$$

- * Widely used in machine learning.



TANH Activation Function

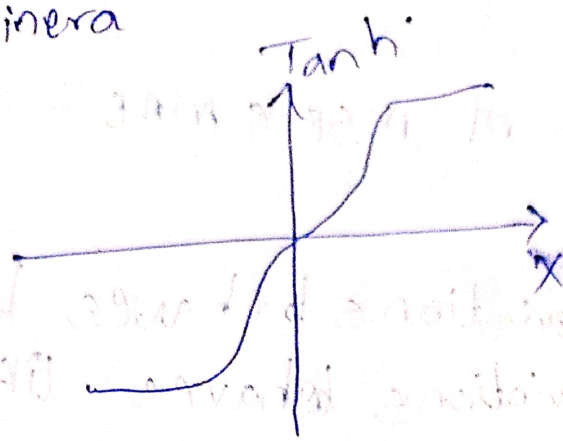
- * This works better than sigmoid function is Tanh function.

- * Tanh (Tangent Hyperbolic function).

$$\tanh(x) = 2 * \text{sigmoid}(2x) - 1$$

- * Ranges from -1 to +1.

* non-linear



Note:-

Use,

→ Sigmoid = binary & multi-label classification.

→ Softmax = multi-class

→ tanh = recurrent neural networks (RNNs).

Loss Function (Cost function).

* It is the error produced by model when comparing the predicted value by the model with the original value given in dataset.

$$\text{Loss} = \text{Actual value} - \text{Predicted value}$$

* By this we can evaluate how our model is working.

Types
Mean Squared Error

$$\text{MSE} = \left(\frac{1}{n}\right) * \sum (y_{\text{actual}} - y_{\text{predicted}})^2$$

mean Absolute Error

$$\text{MAE} = \left(\frac{1}{n}\right) * \sum (y_{\text{actual}} - y_{\text{predicted}})$$

Huber Loss

* Combination of MSE & MAE

* For,

Small Deviations behaves like MSE.

Large Deviations behaves like MAE.

$$\text{Huber Loss} = \left(\frac{1}{n}\right) * \sum \left(\frac{1}{2} * (y_{\text{actual}} - y_{\text{pred}})^2 \right)$$

if $|y_{\text{act}} - y_{\text{pred}}| \leq \delta$
else

$$\frac{1}{n} * \sum \left(\delta * (|y_{\text{act}} - y_{\text{pred}}| - \frac{\delta}{2}) \right)$$

* Huber loss is mainly used when there are more number of outliers.

Binary Cross Entropy (log loss)

* Used in Binary classification problems.

$$\text{BCE} = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

n = no of datapoints.

y_i = actual binary label (0 or 1)

$\hat{y}_i$ = predicted probability.

Categorical Cross Entropy Loss

* multiclass problems.
classification.

$$C.C.E = - \sum_{i=1}^n \sum_{j=1}^K y_{ij} \log(\hat{y}_{ij})$$

n = no of data points.

K = no of classes.

y_{ij} = binary indicator.

$\hat{y}_{ij}$ = predicted probability.

Sparse C.C.E

* Similar to C.C.E loss.

$$S.C.C.E = - \sum_{i=1}^n \log(\hat{y}_i \textcircled{y_i})$$

integer representing
the correct class
for data point i .

Kullback-Leibler divergence loss (KL Divergence)

$$KL \text{ Divergence} = \sum_{i=1}^n \sum_{j=1}^K y_{ij} \log\left(\frac{y_{ij}}{\hat{y}_{ij}}\right)$$

Hinge Loss

Hinge loss is used for training classifiers
especially for SVM's.

$$H.L = \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i \cdot \hat{y}_i)$$

Batch Normalization

* Activations can vary as network weights change, so these large or unstable activation values can make optimization difficult. Hence batch normalization helps keep activations in a more manageable range. As a result training is often faster & more stable.

* BN normalizes activations using statistics from a mini-batch.

* It standardizes the values & then applies learnable scale & shift parameters (γ (gamma) & β (beta)).

* Used in CNN's.

Process

Step 1: - Calculate batch mean

$$\mu_B = (1/m) \sum x_i$$

Step 2: - Calculate batch variance

$$\sigma_B^2 = \left(\frac{1}{m}\right) \cdot \sum (x_i - \mu_B)^2$$

Step 3: - $\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma_B^2 + \epsilon)}$

Step 4:- Learnable scale & shift

$$x_i = \gamma \hat{x}_i + \beta$$

* γ & β are learned during training.

Common CNN pattern

⇒ Convolution → Batch Normalization → ReLU
↓
pooling.

For fully connected layers

⇒ Linear/Dense ⇒ Batch Normalization ⇒
Activation.

Bias & Variance Tradeoff

Bias ⇒ Refers to the error caused by oversimplifying a model.

Variance ⇒ Refers to the error from making the model too sensitive to training data.

High bias ⇒ The model oversimplifies, (underfitting) misses patterns & underfits the data.

Low bias ⇒ The model captures patterns well & is closer to the true values.

Reduce Bias

- 1) Use More Complex Models
- 2) Add relevant features
- 3) Adjust Regularization Strength.

* High variance $\Rightarrow$ The model is too ^{over} (~~under~~ fitting) sensitive to small changes & overfits.

* Low variance $\Rightarrow$ The model is more stable but might miss some patterns.

$$\text{Variance} \Rightarrow E[(f(x) - E f(x))^2]$$

$f(x)$ = predicted value by model

$E f(x)$ = average prediction over multiple training datasets.

Reduce Variance

- 1) Simplify model
- 2) Increase Training Data
- 3) Apply Regularization
- 4) Use Ensemble methods.

Hence, Bias-Variance Tradeoff is the process of balancing bias and variance to achieve the best model performance.

Optimizers

1) Momentum Optimizer

$$V_t = \beta V_{t-1} + (1-\beta) g_t$$

V_t = Velocity

g_t = Current gradient

θ = Model parameters

β = Momentum coefficient

L1 Regularization (LASSO):

* Adds the absolute value of model weights to the loss function.

Advantages

- Performs feature selection
- Reduces overfitting

Disadvantages

- Training can be slower.

Formula $\Rightarrow$ New loss = Original loss + $\lambda \sum |w|$

L2 Regularization (Ridge)

* Adds the square of weights to the loss function.

Advantages

- Stable learning
- Keeps all features

Disadvantages

- * Does not perform feature selection.

Formula $\Rightarrow$ Original loss + $\lambda \sum w^2$

ELASTIC NET

* Combines both L1 & L2 regularization to obtain both feature selection & Stable learning.

Formula $\Rightarrow$ New loss = Original loss + $\lambda_1 \sum |w| + \lambda_2 \sum w^2$

Weight Initialization

* Process of assigning initial values to neural network weights before training.

- $\rightarrow$ Faster convergence
- $\rightarrow$ Prevents vanishing gradients
- $\rightarrow$ Improves final accuracy.
- $\rightarrow$ Prevents exploding gradients.

Methods

1) Zero Initialization

* All weights are initialized to zero.
So, every neuron learns the same features.

2) Random uniform Initialization

* Weights are sampled from a uniform distribution.

3) Random Normal Initialization

*weights are sampled from a Gaussian distribution.

4) Xavier Initialization

*Designed for sigmoid & tanh activations

5) He

*Designed for ReLU

6) LeCun

*Designed for SELU

Convolutional Layer